TEXORA · SESSIONAL NOTES · LAB EXPERIMENT SHEET

Programs with User Defined Functions

Breaking a program into named, testable pieces: writing your own functions with parameters and return values, and understanding exactly what crosses the boundary at a call — copies of values, nothing more.

CSE-2106 · FUNDAMENTALS OF COMPUTING SESSIONAL · EXPERIMENT 8 · LEVEL 2 TERM 1

1. Aim

To design, define and call user-defined functions in C — with parameters, with return values, and with none — and to reuse one function many times in a single program.

2. Concepts you need

- **Why functions:** a task done three times is written once; a named piece can be tested alone; main becomes a readable plan of the program instead of a heap of detail.
- **The three parts:** *declaration (prototype)* — the function's promise: return type, name, parameter types; *definition* — the body that does the work; *call* — using it, e.g. m = max(18, 25);.
- **Parameters:** the values in the call are **actual parameters** (arguments); the variables that receive them inside the function are **formal parameters**. What crosses is a **copy of the value** — this is **call by value**, the only passing C does directly (Experiment 9 examines its limits and the pointer workaround).
- **Return:** a function's type is the type of the value its return hands back; a **void** function returns nothing and is called as a statement.
- **Scope:** variables declared inside a function are **local** — born at the call, gone at the return, invisible elsewhere. Two functions may both have a variable named i without any interference.
- **Placement:** define functions before main, or declare prototypes above main and define below — the compiler must have seen the promise before any call.

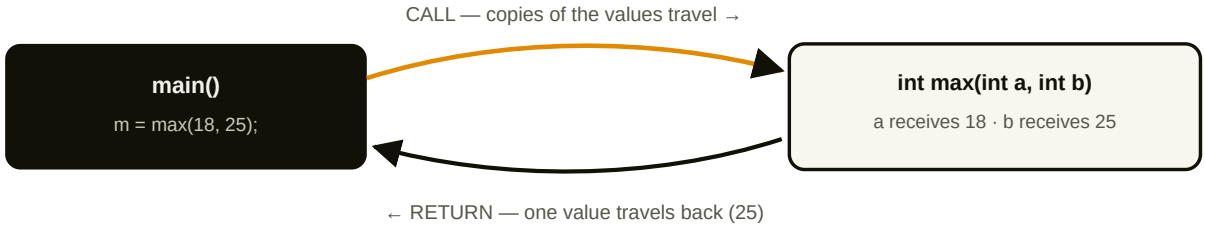


Figure 1 — A function call. Control and copies go out; a single value (or nothing, for void) comes back. Nothing else crosses.

3. Lab procedure

1. Write each program with the functions defined above main; then move one function below main, first without and then with a prototype, and record the compiler's response to each version — the prototype's job made visible.
2. Call max with several pairs, including equal values and negatives; verify each by inspection.
3. In a copy of Example B, print inside the function as well as in main, and observe the order of the lines — evidence of when control passes and returns.

4. Worked examples

A — Two functions, two return types

```
#include <stdio.h>

int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}

float circleArea(float r)
{
    return 3.1416 * r * r;
}

int main(void)
{
    int m;
    float ar;
    m = max(18, 25);
```

```
    ar = circleArea(7);
    printf("Maximum = %d\n", m);
    printf("Circle area = %.2f\n", ar);
    return 0;
}
```

Output:

```
Maximum = 25
Circle area = 153.94
```

Hand check: max(18, 25) — 18 > 25 false, so the else branch returns 25 ✓. circleArea(7) = 3.1416 × 49 = 153.94 ✓ (the same circle as Experiment 4 — now reusable code instead of a repeated calculation).

B — One function called from main: factorial as a service

```
#include <stdio.h>

long factorial(int n)
{
    int i;
    long f = 1;
    for (i = 1; i <= n; i++)
        f = f * i;
    return f;
}

int main(void)
{
    printf("Factorial of 5 = %ld\n", factorial(5));
    return 0;
}
```

Output:

```
Factorial of 5 = 120
```

The loop of Experiment 5 now lives inside a function: main neither knows nor cares how the value is produced. Calling factorial(6) next would cost one line — that is the entire economic argument for functions.

5. Practice tasks

1. Write a function isEven(int n) returning 1 for even and 0 for odd, and a main that reports on 14. Give the code and output.

2. Write a function power(int base, int exp) that returns base raised to exp by repeated multiplication (no library), and use it for 2^5. Give the code, output and the loop trace.
3. Trace this program and give its exact output, explaining the result in terms of call by value:

```
#include <stdio.h>

void inc(int x)
{
    x = x + 1;
}

int main(void)
{
    int a = 5;
    inc(a);
    printf("%d\n", a);
    return 0;
}
```

Answer key

1.
```
#include <stdio.h>

int isEven(int n)
{
    return n % 2 == 0;
}

int main(void)
{
    if (isEven(14))
        printf("14 is even\n");
    else
        printf("14 is odd\n");
    return 0;
}
```

Output: **14 is even**. (Returning the comparison itself — 1 or 0 — is cleaner than an if inside the function doing the same thing.)

2.
```
#include <stdio.h>

long power(int base, int exp)
{
    int i;
    long result = 1;
    for (i = 1; i <= exp; i++)
        result = result * base;
```

```
    return result;
}

int main(void)
{
    printf("2^5 = %ld\n", power(2, 5));
    return 0;
}
```

Trace: result runs 1 → 2 → 4 → 8 → 16 → 32 over i = 1…5. Output: **2^5 = 32**.

3. It prints **5**. inc receives a *copy* of a's value; x = x + 1 raises the copy to 6 inside inc, but inc is void — nothing returns — and the copy is destroyed at the closing brace. main's a was never touched. A function that must change the caller's variable needs the variable's address, which is the subject of Experiment 9.
